

Aplikacija BugBase

Člani

Skupina 22:

- Anže Arhar
- Kristjan Kostanjšek
- Nejc Ločičnik

Povezava do repozitorija

[GitHub povezava](#)

Naslov URL, kjer je aplikacija dostopna

[BugBase povezava](#)

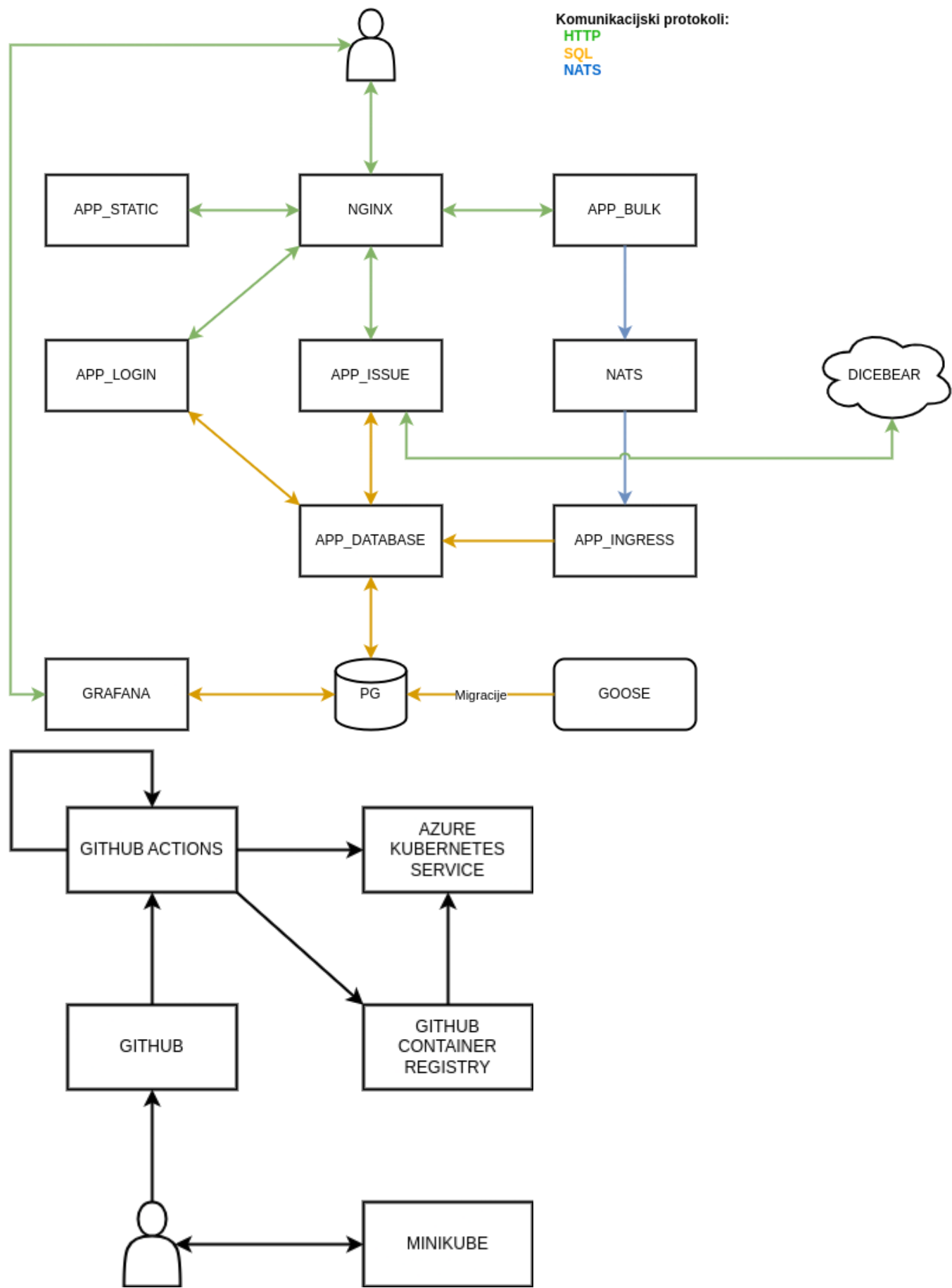
Opis projekta

BugBase je aplikacija za vodenje in upravljanje projektov, zasnovana za razvijalce, ki se pogosto soočajo s kompleksnimi projekti in potrebujejo pomoč pri organizaciji. Poleg registracije in prijave omogoča dodajanje novih projektov in spremljanje njihovega napredka, pri čemer lahko pod vsak projekt dodajamo naloge (issues), ki jih je mogoče povezati z razvojnimi vejami, določiti prioriteto (npr. High, Low) ter spremljati njihovo stanje (npr. Open, Closed). Poleg tega aplikacija podpira dodajanje komentarjev k posameznim nalogam, kar olajša sodelovanje in komunikacijo znotraj ekip.

Ogrodje in razvojno okolje

V skopu razvoja, izdelave in zagona aplikacije smo uporabili naslednja programska ogrodja, orodja, razvojna okolja ter storitve. Celotna aplikacija je napisana v programskem jeziku go. Za osnovno delovanje storitev smo uporabili le standardno knjižnico. Dodatno smo namestili štiri knjižnice: [joho/godotenv](#) za branje okoljskih spremenljivk iz `.env` datotek; [lib/pq](#) sql gonilnik za PostgreSQL podatkovno bazo; [golang-jwt/jwt](#) kot osnovno implementacijo JWT standarda; [nats-io/nats.go](#) za pošiljanje in branje sporočil iz NATS strežnika. Za interaktivno delovanje spletnega dela aplikacije (API klici in preusmeritve uporabnikov) smo uporabili ogrodje [htmx](#). Orodje [make](#) smo uporabili za organizacijo in uporabo pogosto uporabljenih ukazov (postavljanje razvojnega okolja, migracije podatkovne baze, ...). Z orodjem [air](#) smo lokalno poganjali strežnike. Migracije (in testne podatke) na podatkovni bazi smo upravljali z orodjem [goose](#). Za iskanje napak v aplikaciji smo uporabljali orodje [delve](#). Vsako storitev smo pretvorili v samostojno datoteko in jo namestili v lasten Docker vsebnik. S Kubernetesom smo zagotovili visoko razpoložljivost izdelanih vsebnikov. Lokalno testiranje konfiguracije Kubernetesa znotraj okolja Docker nam je omogočilo orodje [minikube](#). Za komunikacijo med nekaterimi storitvami smo tudi uporabili [NATS](#). Za spremljanje osnovnih statistik podatkovne baze smo namestili orodje [Grafana](#). Izdelavo `CHANGELOG.md` datoteke, kjer so zapisane ključne spremembe izvirne kode, je omogočilo orodje [git-cliff](#). Vso izvirno kodo smo hranili na platformi GitHub. Tam smo tudi izvajali zvezno integracijo in zvezno dostavo s GitHub actions. Izdelane Docker vsebnike smo hranili na GitHub-ovi platformi GitHub Container Registry. Vse verzije aplikacije smo poganjali na Azure Kubernetes Service.

Shema arhitecture



Seznam funkcionalnosti mikrorstovitev

V pričujočem poglavju bomo za vsako mikrorstovitev navedli funkcionalnosti, ki jih implementira.

database

Po meri nadgrajena slika PostgreSQL podatkovne baze. Ob zagonu se avtomatično namestijo ključne nadgradnje (migracije) podatkovne baze. Prav tako smo implementirali avtomatično namestitev testnih podatkov.

app-static

Mikrorstovitev deluje kot datotečni strežnik z vlogo dostave skupnih statičnih javascript, CSS in slikovnih datotek, ki jih uporablja več kot ena mikrorstovitev.

app-issue

Mikrorstovitev za prikazovanje in dodajanje vsebine. Na primer: prikaz seznama in dodajanje projektov, prikaz seznama in dodajanje težav posameznega projekta, prikaz in dodajanje komentarjev posamezne težave.

app-login

Mikrorstovitev za prijavo in registracijo uporabnikov. Za avtentikacijo uporabnikov uporabljamo JWT žetone.

app-bulk

Mikrorstovitev omogoča prejemanje večje količine podatkov (novih projektov in nalog) preko POST zahtevka in pošiljanje prejetih podatkov mikrorstovitvi app-ingress preko sporočilnega sistema NATS

app-ingress

Mikrorstovitev omogoča prejem podatkov (novih projektov in nalog) od mikrorstovitve app-bulk preko sporočilnega sistema NATS in vstavljanje prejetih podatkov v podatkovno bazo.

Primeri uporabe

Aplikacija podpira številne primere uporabe. Uporabniki se lahko registrirajo ali prijavijo z obstoječim računom, pregledujejo sezname projektov, nalog, ki spadajo pod posamezne projekte, ter komentarje, povezane z njimi. Poleg tega lahko ustvarjajo nove projekte, naloge in komentarje, dostopajo do svoje profilne strani ter se po potrebi odjavijo iz aplikacije.

Aplikacija podpira tudi kompleksnejši primer uporabe, pri kateri sodelujeta dve mikrorstovitvi. V podatkovno bazo je mogoče preko POST zahtevka uvoziti večje količine podatkov (projektov in nalog). Mikrorstovitev app-bulk prejme zahtevek in podatke posreduje mikrorstovitvi app-ingress prek sporočilnega sistema NATS. Mikrorstovitev app-ingress nato poskrbi za shranjevanje teh podatkov v podatkovno bazo.

Seznam vključenih zahtev

V pričujočem poglavju bomo navedli in na kratko opisali vsako izmed zahtev, ki smo jo implementirali pri našem projektu.

1. Repozitorij

Za razvoj aplikacije smo uporabili GitHub repozitorij, ki smo ga ustrezno strukturirali in opremili za lažjo uporabo. V repozitorij smo vključili datoteko README, ki vsebuje podrobna navodila za lokalno namestitev projekta. Uporabili smo strategijo razvejanja, pri čemer smo ustvarili ključne veje, kot so main in dev. Poleg tega smo za organizacijo, usklajevanje, deljenje dela med člani ekipe in inspiracijo uporabljali GitHub Issues.

2. Mikrostoritve in "cloud-native aplikacija"

Aplikacija je zasnovana za delovanje v oblačnem okolju, torej aplikacijo smo ločili na šibko sklopljene mikrostoritve, katere poganjamo v Docker vsebnikih in orkestriramo z orodjem Kubernetes. Aplikacijo sestavlja šest mikrostoritev, katere smo omenili in opisali v poglavju Seznam funkcionalnosti mikrostoritev.

3. Dokumentacija

V GitHub repozitorij smo vključili markdown datoteko README, ki vsebuje vsa navodila za lokano namestitev aplikacije. Prav tako smo vsaki izmed uporabljenih mikrostoritev dodali README datoteko, ki vsebuje navodila za lokalno namestitev in zagon zgolj te komponente.

4. Dokumentacija API

Po OpenAPI standardu smo opisali končne točke storitev s katerimi uporabnik direktno komunicira (app-login, app-issue, app-bulk). OpenAPI YAML datoteke vključujejo primere vrnjenih podatkov ob pravilni in nepravilni uporabi. Primer:

```
openapi: 3.0.3
info:
  title: App Issue API
  description: API for managing projects, issues, and health checks.
  version: 1.0.0
paths:
  /issue/:
    get:
      summary: Get welcome page
      description: Returns the welcome page.
      responses:
        '200':
          description: HTML welcome page.
          content:
            text/html:
              schema:
                type: string
                example: |
                  <h1 style="text-align: center; font-size: 6rem; color: #10b981;
text-shadow: 5px 5px 2px #104433;">BugBase</h1>
```

```
'400':  
  description: Invalid request.  
  content:  
    text/plain:  
      schema:  
        type: string  
        example: "Invalid Request"
```

5. Cevovod CI/CD

Za CI/CD cevovod smo uporabili GitHub Actions. Tam smo namestili formatiranje kode in izvajanje funkcionalnih testov. Na koncu smo dodali še cevovode za izdelavo Docker vsebnikov in avtomatično namestitev aplikacije v AKS. Za boljši pregled smo v naslovno datoteko README vključili prikaz stanj vseh potekov dela.

6. Helm charts

Za namestitev naših storitev smo uporabili Helm Charte naslednje strukture:

```
├─ Chart.yaml  
├─ ingress.yaml  
├─ templates  
│   ├── configmap.yaml  
│   ├── deployment.yaml  
│   ├── grafana.yaml  
│   ├── nats.yaml  
│   ├── pv.yaml  
│   ├── pvc.yaml  
│   ├── secret.yaml  
│   └─ service.yaml  
├─ values-dev.yaml  
└─ values-prod.yaml
```

Uporabljali smo ločene vrednosti (values.yaml datoteke) za razvijalno in produkcijsko okolje.

7. Namestitev v oblak

Aplikacijo smo namestili v Microsoftovo oblačno platformo Azure. Tam smo uporabili Azure Kubernetes Service.

8. "Serverless" funkcija

Serverless funkcijo smo uporabili za odjavo, ki je originalno bila del app-login storitve. Uporabili smo Azure Function App ("Consumption" opcija) in sledečo funkcijo:

```
import azure.functions as func  
import logging
```

```
app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

@app.route(route="http_logout")
def http_logout(req: func.HttpRequest) -> func.HttpResponse:
    logging.info('Processing logout request.')

    # Create a response object
    response = func.HttpResponse(
        "You have been logged out successfully.",
        status_code=200
    )

    # Set the cookie to clear the JWT
    response.headers.add("Set-Cookie",
        "bugbase_session=";
        "Path=/;"
        "HttpOnly;"
        "SameSite=Strict;"
        "Max-Age=0;") # Max-Age=0 effectively deletes the
cookie

    # Return the response with the cookie header
    return response
```

9. Zunanji API

Za zunanji API smo uporabili [Dicebear](#) za unikatno generacijo uporabniških profilnih slik na podlagi njihovih podatkov (npr. ime, e-mail). Dicebear API je odprt in ne potrebuje avtentikacije.

10. Večnajemništvo

Večnajemništvo smo na podlagi Helm Chartov naredili tako, da smo vsako okolje namestili na ločen imenski prostor in ingress usmerjanje ročno popravili. To deluje iz vidika, da imamo popolnoma ločene namestitve, je pa način kako smo rešili usmerjanje (ingress) napačen oz. ima omejeno funkcionalnost.

11. Preverjanje zdravja

Vsaka izdelana mikrostoritev podpira preverjanje zdravja z API dostopom. Mikrostoritve, ki so povezane s podatkovno bazo, vključujejo preverjanje stanje povezave s podatkovno bazo. Ostale mikrostoritve vračajo le izključno svoje stanje storitve.

```
func (h *HealthHandler) GetHealthLive(w http.ResponseWriter, r *http.Request)
error {
    if err := h.Db.Ping(); err != nil {
        return err
    }
    w.WriteHeader(http.StatusOK)
    return nil
}
```

12. Sporočilni sistemi

V aplikaciji uporabljamo sporočilni sistem NATS, zaradi njegove lahkosti in preprostosti uporabe. Ob zagonu aplikacije se zažene NATS strežnik, na katerega se povežeta mikrostoritvi app-bulk in app-ingress. App-bulk deluje kot producent in objavlja sporočila. App-ingress pa deluje kot prejemnik in se naroči na teme, preko katerih prejema sporočila. Prilagamo še izseke programske kode, kjer vzpostavimo povezavo z NATS strežnikom, objavimo sporočilo s sistemom NATS in se naročimo na sporočila s sistemom NATS.

```
// Initialize NATS connection
func InitNATS() error {
    var err error
    natsUrl := os.Getenv("URL_NATS")
    nc, err = nats.Connect(natsUrl)
    if err != nil {
        return err
    }
    return nil
}
```

```
err = PublishMessage("app.ingress.project", projectData)
if err != nil {
    return fmt.Errorf("error sending project data to NATS: %w", err)
}
```

```
// Subscribe to the project creation topic
_, err := nc.Subscribe("app.ingress.project", func(msg *nats.Msg) {
    var project ProjectInput
    err := json.Unmarshal(msg.Data, &project)
    if err != nil {
        log.Printf("Error unmarshaling project data: %v", err)
        return
    }

    // Insert the project into the database
    AddProjectToDB(db, project)
})
```

13. Zbiranje metrik

Za zbiranje ključnih metrik aplikacije smo postavili instanco Grafane. To smo povezali s podatkovno bazo aplikacije. V uporabniškem vmesniku prikazujemo ključne podatke stanja aplikacije, kot je število registriranih uporabnikov ali zgodovine nalog.

14. Izolacija in toleranca napak

Za bolj zanesljivo delovanje aplikacije smo vključili posredniške API funkcije, ki omogočajo zgodnjo prekinitvev zahtevkov (timeout) in omejitev števila neuspešnih poskusov (retry). Ker je implementacija obeh postopkov relativno enostavna, pri izdelavi nismo uporabili nobenih zunanjih knjižnic.

15. Upravljanje s konfiguracijo

S konfiguracijo upravljamo s ConfigMaps-i (okoljske spremenljivke) in Secrets (geslo baze, JWT skrivnosti). Oboje je del Helm Chart-a tako, da lahko posamezne konfiguracijske spremenljivke spreminjamo direktno s Helm ukazi.

16. Grafični vmesnik

Za aplikacijo smo razvili grafični vmesnik in implementirali podstrani kot so: domača stran, prijavna stran, profilna stran, stran za pregled obstoječih projektov... Za izdelavo grafičnega vmesnika nismo uporabili nobenih orodij, temveč smo ga izdelali samostojno. Za povezavo sprednjega dela z zalednim smo uporabili HTMX, ki omogoča vračanje predlog podatkov v obliki HTML, brez potrebe po nadaljnjem urejanju. Na koncu poročila prilagamo še nekaj primerov GUI nekaterih izmed podstrani naše aplikacije.

17. Ingress Controller

Za dostop do storitev v okolju Kubernetes smo uporabili aplikacijo NGINX. Tam smo namestili usmerjanje zahtevkov do mikrororitev glede na predpono poti vsakega API zahtevka.

Login

E-mail

E-mail

Password

Password

Log in

[Register](#)

User Profile



Name: Vestni

Surname: Študent

Role: Editor

Email: vestni@student@gmail.com

Projects

Add New Project

Title
Best app ever
School project
Android game
Test
Project 1
Project 2
Web Calendar App
Project 100
Project Hello world
Project Zagovor RSO
Zbogom in hvala za vse riše

HomeProjects

ProfileLogout

Best app ever

Issues

Add New Issue

Title	Description	Branch	Status	Priority
asdf	asdf	main	Open	Critical
Profile picture	Allow users to set a picture	main	Open	Low
Login bug	User unable to log in	main	Planning	Critical
UI improvement	Update button color	main	Open	Medium

BugBaseAvite Arhar, Krotjan Kostanjek, Najc Loščnik

HomeProjects

ProfileLogout

Login bug

User unable to log in

Branch: [main](#)

Status: Planning

Priority: Critical

Created at: 2024-10-24T15:46:13Z

Comments

Add New Comment

Comment	Date
Additional details provided	2024-10-24T16:00:00Z
Initial bug report	2024-10-24T15:46:13Z

[Best app ever](#)

BugBaseAvite Arhar, Krotjan Kostanjek, Najc Loščnik